

Your First Queries

Series: ONBRD | **Notebook:** 8 of 10 | **Created:** December 2025

Learning DQL Fundamentals

Dynatrace Query Language (DQL) is how you access data in Grail. This notebook introduces the core concepts and patterns you'll use daily.

Table of Contents

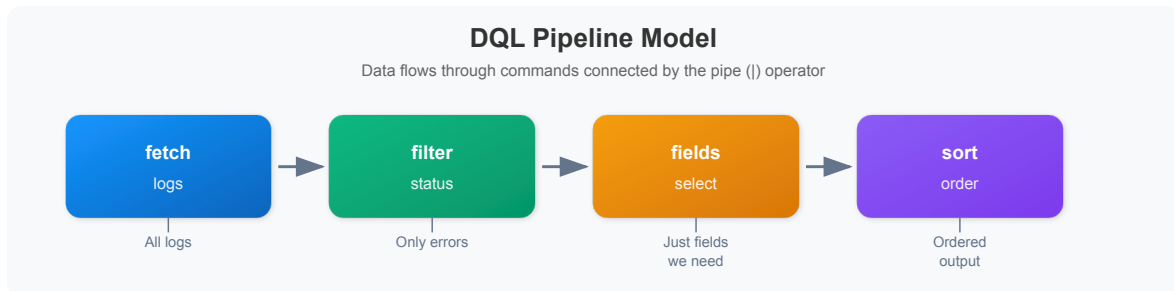
1. DQL Basics
 2. The Pipeline Model
 3. Fetching Data
 4. Filtering
 5. Selecting Fields
 6. Aggregating with Summarize
 7. Sorting and Limiting
 8. Time Ranges
 9. Common Patterns
 10. Next Steps
-

Prerequisites

- Dynatrace environment with data (ONBRD-05, ONBRD-06, ONBRD-07)
- DQL query permissions
- Access to Notebooks or the DQL query interface

1. DQL Basics

DQL is a **pipeline-based query language**—not SQL. Data flows through a series of commands connected by the pipe (`|`) operator.



DQL vs SQL

DQL	SQL	Note
<code>fetch logs</code>	<code>SELECT * FROM logs</code>	Start with data source
<code>\ filter x == "y"</code>	<code>WHERE x = 'y'</code>	Use <code>==</code> , double quotes
<code>\ fields a, b</code>	<code>SELECT a, b</code>	Field selection after fetch
<code>\ summarize count()</code>	<code>SELECT COUNT(*)</code>	Aggregation command
<code>by: {field}</code>	<code>GROUP BY field</code>	Grouping syntax
<code>{"a", "b"}</code>	<code>('a', 'b')</code>	Array syntax (curly braces)

2. The Pipeline Model

Each command in the pipeline operates on the output of the previous command:

```
fetch logs                // 1. Get all logs
| filter loglevel == "error" // 2. Keep only errors
| filter timestamp >= now() - 1h // 3. Last hour only
| fields timestamp, content // 4. Select columns
| sort timestamp desc      // 5. Order by time
| limit 100               // 6. Take first 100
```

Order Matters

- **Filter early** - Reduces data before expensive operations
- **Select fields** - Reduces memory usage
- **Aggregate** - Summarize before sorting
- **Sort** - Order the final results
- **Limit** - Control output size

3. Fetching Data

Every DQL query starts with a `fetch` command specifying the data source.

```
// Fetch logs (most common)
fetch logs
| limit 10
```

```
// Fetch spans (distributed traces)
fetch spans
| limit 10
```

```
// Fetch entity data (hosts)
fetch dt.entity.host
| limit 10
```

```
// Fetch problems
fetch dt.davis.problems
| limit 10
```

Common Data Sources

Source	Description
logs	Log records
spans	Distributed trace spans
events	System events
bizevents	Business events
dt.entity.host	Host entities
dt.entity.service	Service entities
dt.entity.process_group	Process group entities
dt.davis.problems	Detected problems

4. Filtering

Use `filter` to narrow results. Filter as early as possible for performance.

```
// Filter by equality
fetch logs
| filter loglevel == "error"
| limit 20
```

```
// Filter with multiple conditions (AND)
fetch logs
| filter loglevel == "error"
| filter timestamp > now() - 1h
| limit 20
```

```
// Filter with OR condition
fetch logs
| filter loglevel == "error" or loglevel == "warn"
| limit 20
```

```
// Filter using IN for multiple values
fetch logs
| filter in(loglevel, {"error", "warn", "fatal"})
| limit 20
```

```
// Filter with string matching
fetch logs
| filter contains(content, "timeout")
| limit 20
```

Filter Operators

Operator	Example	Description
<code>==</code>	<code>field == "value"</code>	Equals
<code>!=</code>	<code>field != "value"</code>	Not equals
<code>></code> , <code><</code>	<code>count > 10</code>	Greater/less than
<code>>=</code> , <code><=</code>	<code>count >= 10</code>	Greater/less or equal
<code>and</code> , <code>or</code>	<code>a == 1 and b == 2</code>	Logical operators
<code>in()</code>	<code>in(field, {"a", "b"})</code>	Value in set
<code>contains()</code>	<code>contains(field, "text")</code>	Substring match
<code>isNull()</code>	<code>isNull(field)</code>	Field is null
<code>isNotNull()</code>	<code>isNotNull(field)</code>	Field is not null

5. Selecting Fields

Use `fields` to select specific columns. This improves readability and performance.

```
// Select specific fields from logs
fetch logs
| fields timestamp, loglevel, log.source, content
| limit 20
```

```
// Create calculated fields
fetch spans
| fields span.name,
        duration,
        duration_ms = duration / 1000000.0
| limit 20
```

```
// Rename fields with aliases
fetch dt.entity.host
| fields name = entity.name,
        status = state,
        os = osType
| limit 20
```

fieldsAdd vs fields

Command	Effect
<code>fields</code>	Keeps only specified fields
<code>fieldsAdd</code>	Adds new fields, keeps all existing

```
// fieldsAdd keeps existing fields and adds new ones
fetch spans
| fieldsAdd duration_ms = duration / 1000000.0
| fields span.name, duration, duration_ms
| limit 10
```

6. Aggregating with Summarize

Use `summarize` to aggregate data. Combine with `by:` for grouping.

```
// Simple count
fetch logs, from: now() - 1h
| summarize total_logs = count()
```

```
// Count by group
fetch logs, from: now() - 1h
| summarize log_count = count(), by: {loglevel}
| sort log_count desc
```

```
// Multiple aggregations
fetch spans, from: now() - 1h
| filter span.kind == "server"
| summarize
    request_count = count(),
    avg_duration = avg(duration),
    max_duration = max(duration),
    by: {service.name}
| sort request_count desc
| limit 20
```

```
// Conditional counting
fetch spans, from: now() - 1h
| filter span.kind == "server"
| summarize
    total = count(),
    errors = countIf(span.status_code == "error"),
    by: {service.name}
| fieldsAdd error_rate = 100.0 * errors / total
| sort error_rate desc
| limit 20
```

Common Aggregation Functions

Function	Description
<code>count()</code>	Count records
<code>countIf(condition)</code>	Count where condition is true
<code>sum(field)</code>	Sum values
<code>avg(field)</code>	Average value
<code>min(field)</code>	Minimum value
<code>max(field)</code>	Maximum value
<code>percentile(field, 95)</code>	95th percentile

7. Sorting and Limiting

Control output order and size.

```
// Sort descending (newest first)
fetch logs, from: now() - 1h
| fields timestamp, loglevel, content
| sort timestamp desc
| limit 20
```

```
// Sort by multiple fields
fetch spans, from: now() - 1h
| filter span.kind == "server"
| summarize request_count = count(), by: {service.name}
| sort request_count desc
| limit 10
```

```
// Find slowest spans
fetch spans, from: now() - 1h
| fields span.name, service.name, duration
| sort duration desc
| limit 10
```

8. Time Ranges

Control the time range for your queries.

```
// Last hour (using from: parameter)
fetch logs, from: now() - 1h
| summarize count()
```

```
// Specific time range
fetch logs, from: now() - 24h, to: now() - 12h
| summarize count()
```

```
// Last 7 days
fetch dt.davis.problems, from: now() - 7d
| summarize problem_count = count(), by: {status}
```


Time Units

Unit	Example	Description
s	<code>now() - 30s</code>	Seconds
m	<code>now() - 15m</code>	Minutes
h	<code>now() - 2h</code>	Hours
d	<code>now() - 7d</code>	Days

9. Common Patterns

Here are patterns you'll use frequently.

Error Investigation

```
// Find error logs with context
fetch logs, from: now() - 1h
| filter loglevel == "error"
| fields timestamp, log.source, content
| sort timestamp desc
| limit 50
```

Service Performance

```
// Service response time summary
fetch spans, from: now() - 1h
| filter span.kind == "server"
| summarize
  requests = count(),
  avg_ms = avg(duration) / 1000000.0,
  p95_ms = percentile(duration, 95) / 1000000.0,
  by: {service.name}
| sort requests desc
| limit 20
```

Log Volume Analysis

```
// Log volume by source and severity
fetch logs, from: now() - 1h
| summarize log_count = count(), by: {log.source, loglevel}
| sort log_count desc
| limit 30
```

Entity Inventory

```
// Host inventory with details
fetch dt.entity.host
| fields
    name = entity.name,
    state,
    os = osType,
    cores = cpuCores
| sort name
| limit 50
```

10. Next Steps

With DQL fundamentals covered:

1. **ONBRD-09: Setting Up Alerts** - Configure alerting and notifications
2. Practice with your own data
3. Explore the DQL documentation for advanced functions
4. Try creating queries in Notebooks

DQL Checklist

- ☐ Understand the pipeline model
- ☐ Can fetch different data types
- ☐ Can filter with conditions
- ☐ Can select and calculate fields
- ☐ Can aggregate with summarize
- ☐ Can sort and limit results

- [] Can specify time ranges
-

Summary

In this notebook, you learned:

- DQL uses a pipeline model (not SQL)
 - `fetch` starts every query
 - `filter` narrows results
 - `fields` selects and calculates columns
 - `summarize` aggregates data
 - `sort` and `limit` control output
 - Time ranges with `from:` and `to:`
-

References

- [DQL Reference](#)
 - [DQL Functions](#)
 - [DQL Commands](#)
 - [DQL Examples](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.